CHAPTER 1

DESIGN PATTERNS

1 Introduction to the Design Patterns . 2
2 Classification of Design Patterns . . 3
3 Creational Design Patterns 5
4 Structural Design Patterns 7
5 Behavioral Design Patterns 10
6 Architectural Design Patterns 14
7 Quiz: Design Patterns 17

This chapter teaches you about the classification of design patterns, including creational, structural, and behavioral patterns, followed by a quiz.

1 Introduction to the Design Patterns

1.1 What are design patterns?

Designing efficient and reusable code is something that all developers strive for. When working on large applications, code structure becomes an integral element where we need to prevent repeating code for similar tasks. However, trying to build upon and adding features to an existing system is quite a challenging task, since a developer needs to know the complete particulars. These include the relationships that each entity possesses or the sort of hierarchy that exists between different entities. Also , developers need to update their code in such a way as to make it easily adaptable to change in the future. In such scenarios, it'll be helpful to have a structure that could be used to solve various common issues. This is where the design patterns come into play.
Design patterns are solutions to frequently occurring real-life problems in software design. They can also be considered customizable templates that can meet the requirements of a particular design problem. Let's take the example of an ice-cream factory that serves as the base of operations to produce various types of ice creams according to the defined requirements. This is similar to how a design pattern can be used as a template in different circumstances.

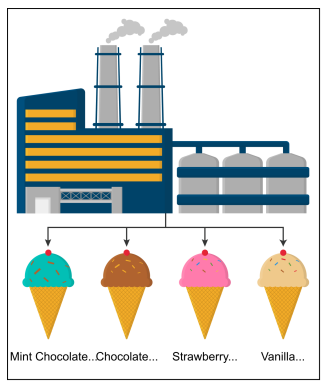


Figure 1.1: A real-life example of the design patterns

1.2 Structure of design patterns

- **Pattern name:** This is an identification step that describes a design problem.
- **Intent:** This step describes the use case of the particular design pattern.
- **Motivation:** This step illustrates the problem and talks about the inner components in the pattern to solve the problem.
- **Structure:** This step visualizes the structure using a graphical representation of the classes in the pattern.
- **Consequences:** This step describes the trade-offs of a particular pattern.
- **Implementation:** This step illustrates an example of code in any popular programming language.

1.3 Advantages of design patterns

The following are some of the advantages of using design patterns:

- They provide correct and efficient solutions since they have been derived and optimized by various experienced programmers over time.

- They are generic templates that can be modified and used for solving different problems.
- They can provide a clean and elegant solution to a large problem by avoiding repetition in the code.
- They provide a template on which the developers can build upon. This allows developers to spend less time on code structure and more on the overall quality of the solution.

1.4 Consequences of unfamiliarity with the design patterns

While there are many advantages of using design patterns, it is important to fully understand the depths of design patterns, since unfamiliarity might cause the following issues:

- Design patterns can complicate the architecture of the application if they are managed poorly.
- Developers who are not familiar with them might end up getting confused as to why certain patterns are being used.

Now that we've gotten a flavor of what design patterns are, let's explore some of their different types in the next lesson.

2 Classification of Design Patterns

The illustration below demonstrates some of the popular design patterns:

Classification of design patterns

[Interactive Mind Map - See online version for interactive visualization]

- **Design Patterns**
  - **Creational Design Patterns**
    - Factory
    - Constructor
    - Singleton
    - Builder
    - Prototype
    - Abstract
  - **Structural Design Patterns**
    - Decorator
    - Facade
    - Adapter
    - Bridge
    - Composite
    - Flyweight
    - Proxy

  - **Behavioral Design Patterns**
    * Chain of Responsibility
    * Command
    * Iterator
    * Mediator
    * Observer
    * Visitor
    * Interpreter Pattern
    * Template Pattern
    * Memento Pattern
    * State Pattern
    * Strategy Pattern

- **Architectural Design Patterns**
  - MVC
  - MVP
  - MVVM
  - Interpreter
  - Memento
  - State
  - Template Method
  - Revealing Module

2.1 Creational design patterns

These patterns are used to provide a mechanism for creating objects in a specific situation without revealing the creation method. The normal approach for creating an object might lead to complexities in the design of a project. These patterns allow flexibility in deciding which objects need to be created for a specific use case by providing control over the creation process.

2.2 Structural design patterns

These patterns are concerned with class/object composition and relationships between objects. They let us add new functionalities to objects so that restructuring some parts of the system does not affect the rest. Hence , when some parts of the structure change, the entire system does not need to change.

2.3 Behavioral design patterns

These patterns are concerned with communication between dissimilar objects in a system. They streamline communication and ensure that the information is synchronized between such objects.

2.4 Architectural design patterns

These patterns are used for solving architectural problems within a given context in software architecture. As discussed above, design patterns play a major role in the structuring of an application and are of great

use to a developer. In the next few lessons, we will discuss the most common design patterns so that you can confidently answer any questions related to them in an interview and be able to implement them at your job.

3 Creational Design Patterns

3.1 Introduction to creational design patterns

In this lesson, we will discuss creational design patterns. **Creational design patterns** deal with object creation mechanisms. As the name implies, these patterns provide optimized object creation techniques. They help cater to the design and complexity problems that might occur when using the basic approach. They also help control the creation of objects.
The chart below shows the patterns that fall under this category:

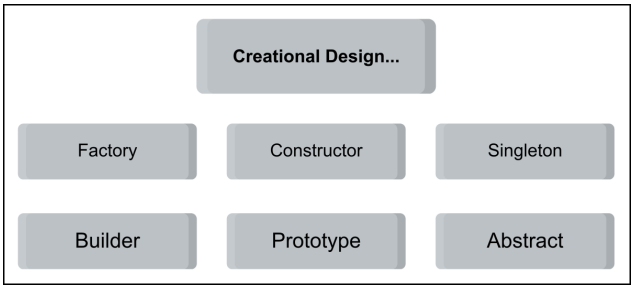


Figure 1.2: Creational design patterns

Factory pattern

The **Factory pattern** is a creational pattern that provides a template that can be used to create objects. It is used in complex situations where the type of the object required varies and needs to be specified in each case.
It does not use the `new` keyword directly to instantiate objects. This means that it does not explicitly require the use of a constructor to create objects. Instead , it provides a generic interface that delegates the object creation responsibility to the corresponding subclass.

Constructor pattern

The **Constructor pattern**, as the name defines, is a class-based pattern that uses the constructors present in the class to create specific types of objects.

Singleton pattern

The **Singleton pattern** is a type of design pattern that restricts the instantiation of a class to a single object. This allows the class to create its instance the first time it is instantiated. However, on the next try, the existing instance of the class is returned. No new instance is created.

Builder pattern

The Builder pattern is a type of a creational design pattern that helps in building complex objects using simpler objects. It provides a flexible and step-by-step approach towards making these objects. It also shields the representation and process of creation.

Prototype pattern

The **Prototype pattern** is used to instantiate objects with some default values using an existing object. It clones the object and provides the existing properties to the cloned object using prototypal inheritance.
In **prototypal inheritance**, a prototype object acts as a blueprint from which other objects inherit when the constructor instantiates them. Hence, any properties defined on the prototype of a constructor function will also be present in the cloned object it creates.

Abstract pattern

We use the Factory pattern to create multiple objects from the same family without having to deal with the creation process. The **Abstract pattern** is similar. The difference is that it provides a constructor to create families of related objects. It is abstract, which means that it does not specify concrete classes or constructors.

When to use creational design patterns?

Creational Design Patterns	When to use
Factory pattern	• When the type of objects required cannot be anticipated beforehand. • When multiple objects that share similar characteristics need to be created. • When you want to generalize the object instantiation process, since the object set up is complex in nature.
Constructor pattern	• You can use it when you want to create multiple instances of the same template, since the instances can share methods but can still be different. • It can be useful in the Libraries and Plugins design.
Singleton pattern	• The Singleton pattern is mostly used in cases where we want a single object to coordinate actions across a system. • **Services** can be singleton since they store the state, configuration, and provide access to resources. Therefore, it makes sense to have a single instance of a service in an application. • **Databases** such as MongoDB utilize the Singleton pattern when it comes to database connections. • **Configurations** are used if there is an object with a specific configuration, and we don't need a new instance every time that configuration object is needed.
Builder pattern	• You can use this design pattern when building apps that require you to create complex objects. It can help you hide the construction process of building these objects. • A good example would be a DOM, where we might need to create plenty of nodes and attributes. The construction process can get quite messy if we are building a complex DOM object. In cases like these, the Builder pattern can be used.
Prototype pattern	• To eliminate the overhead of initializing an object. • When we want the system to be independent about how the products in it are created. • When creating objects from a database whose values are copied to the cloned object.
Abstract pattern	• Applications requiring the reuse or sharing of objects. • Applications with complex logic because they have multiple families of related objects that need to be used together. • When we require object caching. • When the object creation process is to be shielded from the client.

Table 1.1: When to use creational design patterns?

Let's look at the structural design patterns in the next lesson.

4 Structural Design Patterns

4.1 Introduction to structural patterns

In this lesson, we will discuss structural design patterns. As the name implies, these patterns are concerned with object relationships and the structure of classes or objects. They help to add new functionality without having to modify the entire system. They ensure that if one part of a system changes, the whole system does not change with it. Let 's look at the most common structural patterns that are used in solving design problems.

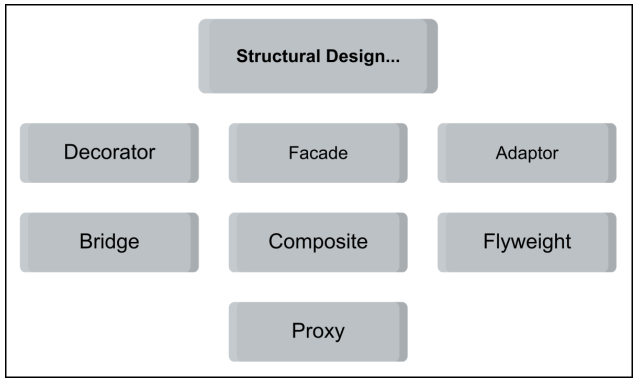


Figure 1.3: Structural design patterns

4.2 Decorator pattern

The **Decorator pattern** focuses on adding properties, functionalities, and behavior to existing classes dynamically. The additional decoration functionalities aren't considered essential enough to be a part of the original class definition since they can cause clutter. Hence, the Decorator pattern lets us modify the code without changing the original class.
Unlike creational patterns, the Decorator pattern is a structural pattern that does not focus on object creation but rather on decoration. Hence , it doesn't rely on prototypal inheritance alone. It takes the object and keeps adding decoration to it. This makes the process more streamlined. Let's look at an example to understand this concept better.
The illustration below shows that ice-cream toppings can be a part of the Decorator pattern for a plain vanilla cone.

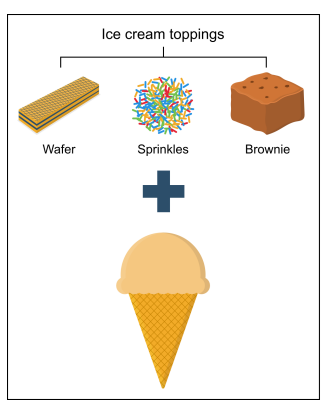


Figure 1.4: Ice-cream toppings can be implemented using the Decorator pattern

4.3 Facade pattern

The word "**facade**" means a deceptive front or appearance. Following this definition, a **Facade pattern** provides a simpler interface that hides the complex functionalities of a system. The Facade pattern allows us to hide all the messy logic from the client and only display the clear and easy-to-use interface to them. This allows the client to interact with an API easily in a less error-prone way and without accessing the inner workings directly.

4.4 Adapter pattern

The **Adapter pattern** allows classes that have different interfaces (properties/methods of an object) to work together. It translates the interface for a class to make it compatible with another class.
This pattern is useful if an API is modified or new implementations are added to it. In this case, if the other parts of a system are still using the old API, the Adapter pattern will translate the interface so that the two can work together. The illustration below demonstrates the use of the Adapter pattern.

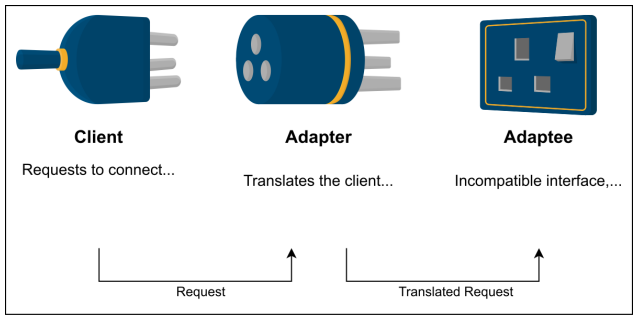


Figure 1.5: Concept for the Adapter pattern

4.5 Bridge pattern

The **Bridge pattern** allows separate components with separate interfaces to work together. It keeps an object's interface separate from its implementation, allowing the two to vary independently.
An example is controlling an air conditioner with a remote. The air conditioners can be of different types and each of them is controlled by a different remote. The remotes can vary, that is, a new one with better features can be introduced, but that won't make any changes to the air conditioner classes. The same goes the other way round. The Bridge pattern allows input and output devices to work together but vary independently.

4.6 Composite pattern

The **Composite pattern** is used to structure objects in a tree-like hierarchy. Here, each node of the tree can be composed of either child node(s) or be a leaf (no children objects). This pattern allows the client to work with these components uniformly, that is, a single object can be treated exactly how a group of objects is treated.
This pattern allows the formation of a deeply-nested structure. If a leaf object receives the request sent by the client, it will handle it. However, if the recipient is composed of children, the request is forwarded to the child components.
A Composite pattern consists of the following:

- **Component:** An abstract class that contains methods such as `add`, `remove`, and `get` that are used in managing the children. The component can be a leaf object or composite.
- **Composite:** This is the subclass that implements a component. It is composed of other components (children).
- **Leaf:** This is the subclass that implements a component. It does not have children.

We can visualize this in the diagram below:

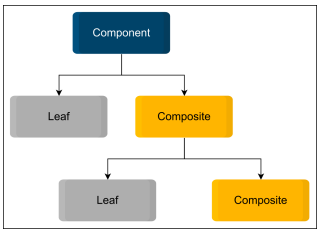


Figure 1.6: Concept for the Composite pattern

4.7 Flyweight pattern

The **Flyweight pattern** focuses on how related objects share data. It helps prevent repetitive code and increases efficiency when it comes to data sharing as well as conserving memory.
This pattern takes the common data structures/objects that are used by a lot of objects and stores them in an external object (flyweight) for sharing. We can say that it is used for caching purposes. So , the same data does not need to have separate copies for each object, instead, it is shared amongst all.
A flyweight is an independent object that can be used in multiple contexts simultaneously. It cannot be distinguished from the instances of objects that are not sharable. A flyweight object can consist of two states:

- **Intrinsic:** This state is stored in the flyweight. It contains the information required by the internal methods of objects. It is independent of the context of the flyweight and is sharable with other objects.
- **Extrinsic:** This state depends on the context of the flyweight and it cannot be shared. Normally, the client objects pass the extrinsic state to the flyweight object when needed.

Let's see a visual depiction of the Flyweight pattern:

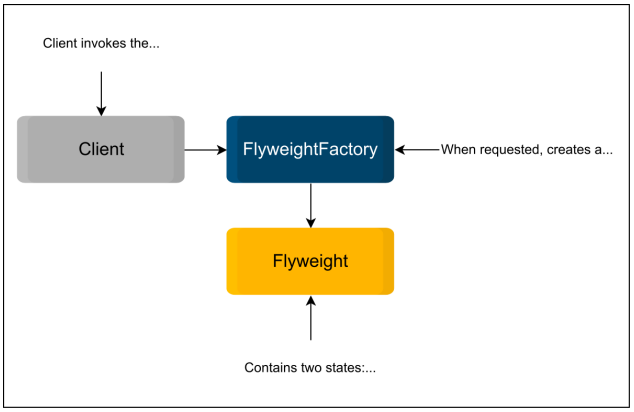


Figure 1.7: Concept for the Flyweight pattern

4.8 Proxy pattern

As the name implies, the **Proxy pattern** is a structural pattern that creates a proxy object. It acts as a placeholder for another object, controlling access to it.
Usually, an object has an interface with several properties/methods that a client can access. However, an object might not be able to deal with the clients' requests alone due to heavy load or constraints such as dependency on a remote source that might cause delays (e.g., network requests). In these situations, adding

a proxy helps in dividing the load with the target object.
The proxy object looks exactly like the target object. A client might not even know that they are accessing the proxy object instead of the target object. The proxy handles the requests from the clients and forwards them to the target object, preventing undue pressure on the target.

4.9 When to use structural patterns?

Let's see when we can use the structural patterns discussed above:

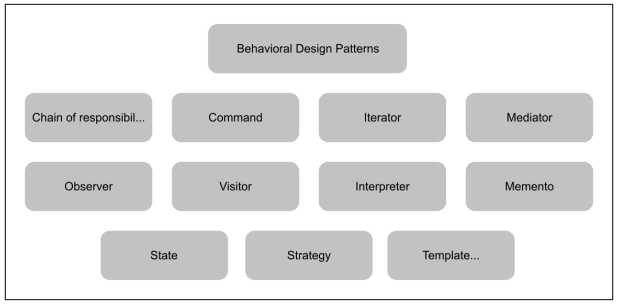
Structural Design Patterns	When to Use
Decorator	• To modify or extend the functionality of an object without changing its base code. • To implement additional functionalities of similar objects instead of reusing the same code.
Facade	• To simplify a client's interaction with a system by hiding the underlying complex code. • To interact with the methods present in a library without knowing the processing that happens in the background.
Adapter	• To enable old APIs to work with new refactored ones. • To allow an object to cooperate with a class that has an incompatible interface. • To reuse the existing functionality of classes.
Bridge	• To extend a class in several independent dimensions. • To change the implementation at run time. • To share the implementation between objects.
Composite	• To allow the reuse of objects without worrying about their compatibility. • To develop a scalable application that uses plenty of objects. • To create a tree-like hierarchy of objects.
Flyweight	• To share a list of immutable strings across the application. • To prevent load time as it allows caching.
Proxy	• To reduce the workload on the target object.

Next, let's look at the behavioral design patterns used in the object-oriented design.

5 Behavioral Design Patterns

5.1 What are behavioral patterns?

Behavioral design patterns define how objects communicate and collaborate to accomplish tasks within a system. These patterns focus on effective interaction and responsibility-sharing among objects, without requiring significant changes to their internal code. This approach makes software systems more flexible, easier to maintain, and adaptable to future requirements.
For example, while designing a notification system, we might want to send an email, a push notification, or a text message whenever a new blog post is published. Instead of writing separate code for each type, we can use the observer pattern, which automatically updates all notification services when something new happens. If we decide to add a Slack message later, we simply introduce a new observer—no changes are needed in the main publishing code.
Several well-known behavioral design patterns exist. The most common patterns include Chain of Responsibility, Command, Iterator, Mediator, Observer, Visitor, Interpreter, Memento, State, Strategy, and Template Method. This lesson will explain the most important and commonly used patterns to help build a strong understanding of practical design and interview preparation.
The illustration below shows all the patterns that fall under this category:



5.2 Chain of Responsibility pattern

The **Chain of Responsibility pattern** allows a request sent by a client to be received by more than one object. It creates a chain of loosely-coupled objects that, upon receiving the request, either handle it or pass it to the next handler object.
A common example of this pattern is event bubbling in the DOM. An event propagates through different nested elements of the DOM until one of them handles it.

5.3 Command pattern

The **Command pattern** allows encapsulation of the requests or operations into separate objects. It decouples the objects that send requests from those responsible for executing them.
Consider an example where the client is accessing the methods of an API directly throughout the application. What will happen if the implementation of that API changes? The change will have to be made everywhere the API is being used. To avoid this, we could use abstraction and separate the objects requesting from those implementing the request. Therefore, only the object making the call will need to change if a change occurs.

5.4 Iterator pattern

The **Iterator pattern** allows the definition of various types of iterators that can sequentially iterate a collection of objects without exposing the underlying form.
Iterators encapsulate how the traversal occurs in an iteration. Most languages have built-in iterators such as IEnumerable andIEnumerator. Iterators follow the behavior where they call a `next` function and step through a set of values until they reach the end. To do this, they need to maintain a reference to the current position and the collection they are traversing. Hence , an iterator has functions such as `next`, `hasNext`, `currentItem`, and`each`.

5.5 Mediator pattern

It is a behavioral pattern that allows a **mediator** (a central authority) to act as the coordinator between different objects instead of the objects referring to each other directly. As the name implies, a mediator is a central authority through which various components can communicate. It allows the loose coupling of objects.
A real-life example is a chat application. Here, the chat box acts as the mediator through which various users interact with one another.

5.6 Observer pattern

The **Observer pattern** is an important behavioral design pattern. It allows objects (observers) who have subscribed to an event to wait for input and react when notified. Therefore, they don't have to continuously check whether the input has been provided. The main subject maintains a list of all the observers, and whenever the event occurs, it notifies the observers so they can update their states accordingly.
Let's look at a real-life example to map to this pattern. Consider a website that posts interesting articles. Every day, you visit the site to check for new articles; if there are none, you revisit after some time. What if

you get a subscription to the website instead? Once you have the subscription, you'll be notified whenever a new article is posted. So now, instead of checking the site every few hours, you just wait for the notification about a new article.

5.7 Visitor pattern

The **Visitor pattern** allows the definition of new operations on the collection of objects without changing the objects' structure. This allows us to separate the class from the logic it implements.
The extra operations can be encapsulated in a visitor object. The objects can have a visit method that accepts the visitor object. The visitor can then make the required changes and perform the operations on the received object. This allows the developers to make future extensions, extend the libraries/frameworks, etc.

5.8 Interpreter pattern

The **Interpreter pattern** provides a way to interpret or translate sentences in a simple language using a set of interpreter objects. Each interpreter object represents a rule or part of the language grammar. Together, these objects can read, understand, and process statements. This pattern is helpful when defining and processing simple languages or expressions.
A real-life example is a simple calculator. Here , we can create different `Expression` objects like `NumberExpression` for numbers and `AddExpression` or `SubtractExpression` for operations. Each expression object knows how to interpret itself and returns a result.

5.9 Memento pattern

The **Memento pattern** captures and externalizes an object's internal state so that it can be restored to this state later without violating encapsulation. This pattern is commonly used to implement undo/redo functionality.
This pattern uses three types of objects:

1. **Originator:** The object whose state needs saving/restoring
2. **Memento:** The object that stores the state
3. **Caretaker:** The object that requests and holds mementos, but does not modify them

For example, in a text editor, the `TextEditor` object is the `Originator`. When we want to save our work, the `TextEditor` creates a Memento object that stores the current text. The History object is the `Caretaker`. It keeps a list of Memento objects so we can undo changes. If we want to undo, the `TextEditor` asks for the last Memento and restores its state.

5.10 State pattern

The **State pattern** lets an object change its behavior when its internal state changes. We create different state objects instead of using many `if` or `switch` statements. The main object keeps a reference to its current state object and passes work to it. Each state object defines how to handle actions for that state.
For example, in a music player, the `MusicPlayer` object can be in a `PlayingState` or a `PausedState`. When we press the play/pause button, the `MusicPlayer` asks its current state object what to do. If it is in `PlayingState`, the state object will pause the music and change the state to `PausedState`. If it is in `PausedState`, the state object will play the music and switch back to `PlayingState`.

5.11 Strategy pattern

The **Strategy pattern** allows us to define a family of related algorithms and put each in its strategy object. A context object holds a reference to a strategy object and delegates the work to it instead of handling the logic directly. This pattern makes changing an algorithm at runtime easy without changing the main object's

code.
A real-life example is a navigation application. Here , the `NavigationApp` object acts as the context and selects a `RouteStrategy` object, such as `FastestRoute`, `ShortestRoute`, or `ScenicRoute`, to calculate the route. The navigation app can switch to a different strategy at any time to find a route in another way.

5.12 Template Method pattern

The **Template Method pattern** defines the basic steps of an algorithm in a base class, with some steps fixed and others left for subclasses to fill in. This lets us keep the process's common structure in one place while allowing parts to vary when needed. It helps enforce consistency and reuse across related tasks.
A real-life example is a system for making beverages. The `BeverageMaker` base class defines how to make a drink: boil water, brew, pour into a cup, and add extras. Subclasses like `TeaMaker` and `CoffeeMaker` provide their own way to brew and add extras, while the main steps stay the same.

5.13 When to use behavioral design patterns

Behavioral design patterns are useful in different situations. Here are some examples of when we should use each pattern.

Pattern	Where It Can Be Used
Chain of Responsibility	
	• Used when different requests require different handlers. • Used in DOM event bubbling.
Command	
	• Queueing and executing commands later. • Implementing reset/undo operations. • Keeping a history of requests.
Iterator	
	• Accessing collection elements without exposing representation. • Creating generic iterators for any collection type.
Mediator	
	• Avoiding tight coupling between many interacting objects. • Improving readability and maintainability.
Observer	
	• Creating loosely-coupled one-to-many relationships. • Notifying multiple components of state changes.

Pattern	Where It Can Be Used
Visitor	
	• Adding new operations without modifying existing classes.
	• Operating on complex object structures.
Interpreter	
	• Implementing small languages or DSLs.
	• Parsing rules or expression grammars.
	• Interpreting math expressions or config scripts.
Memento	
	• Undo/redo implementations.
	• Saving snapshots of object state.
State	
	• Changing behavior based on internal state.
	• Removing complex conditional logic.
Strategy	
	• Selecting algorithms at runtime.
	• Swapping business rules without modifying main logic.
Template Method	
	• Sharing algorithm structure while allowing subclass variation.
	• Frameworks where subclasses overwrite specific steps.

Now that we have reviewed all the design patterns, let's test the concepts we have learned in the next quiz lesson.

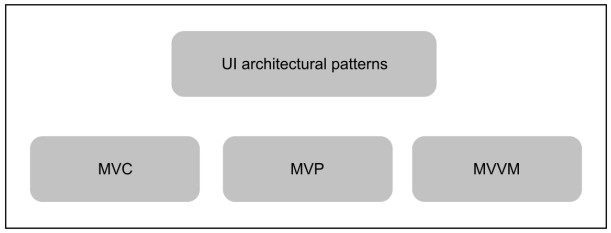
6 Architectural Design Patterns

6.1 What are architectural design patterns?

Architectural design patterns help us decide how to structure an entire application or its large parts. These patterns give us a blueprint for organizing code to stay clean, flexible, and easy to maintain as the project grows. There are two main types of architectural patterns:

- **UI architectural patterns:** These describe how to organize code inside the presentation layer(It is the application part that shows the user interface and handles user interactions.) of an app. They help separate the user interface, business logic, and data, making testing and updating each part easier without breaking the others. Some commonly used UI architectural patterns include MVC, MVP, and MVVM.
- **System-level architectural patterns** describe how the major parts of an application work together. Some common examples include layered architecture, microservices, client–server, and event-driven architecture.

This lesson will focus on three popular UI architectural patterns: MVC, MVP, and MVVM. These patterns are widely used in web, desktop, and mobile app development.



6.2 Model-View-Controller (MVC)

MVC (Model-View-Controller) is a widely used architectural pattern in software design, especially for building user interfaces and web applications. It separates an application into three interconnected components:

- **Model:** It handles the data and business logic of the application. It is responsible for managing the rules of the application, storing and retrieving data from a database, and performing any calculations or operations needed. The Model does not care how the data is displayed; it only cares about the data and rules.
- **View:** It is the user interface that shows the data. It displays the data provided by the Model and presents it in a format that is easy for users to read and interact with. The View does not store any data by itself; it just shows what the Model gives it. If the data changes, the View updates to show the new information.
- **Controller:** It handles user input and interactions and bridges the View and the Model. When a user clicks a button or submits a form, the Controller takes this input, tells the Model what to do with it (like saving new data), and then decides which View should be shown next or how the View should be updated.

In a web app, the HTML page is the View, the database and business rules are the Model, and the routes or handlers are the Controller. When a user clicks a button, the Controller updates the Model, and the View shows the latest data. This pattern is helpful when we want to keep UI and logic separate so that we can update one without affecting the other.

6.3 Model-View-Presenter (MVP)

The **Model-View-Presenter (MVP)** pattern is similar to MVC but replaces the Controller with a **Presenter**. The Presenter handles all the logic for the View.
In MVP:

- **Model:** It holds the data and contains the business logic. It manages the application's data, rules, and operations like in MVC. The Model does not know how the data is shown to the user.

- **View:** It is the user interface. It displays the data to the user and sends user actions (like button clicks or form submissions) to the Presenter. The View is usually kept as simple as possible and contains no business logic.
- **Presenter:** It is the middle layer between the View and the Model. It talks to the Model to get the required data and then updates the View with that data. The Presenter also handles user input by deciding what to do when the user interacts with the View. Unlike the Controller in MVC, the Presenter usually has more control over the View and can tell it exactly what to display.

In an Android app, the screen is the View, the data classes are the Model, and the Presenter handles what happens when the user clicks a button or enters text. This pattern is useful when writing unit tests for the logic without testing the actual UI.

6.4 Model-View-ViewModel (MVVM)

The **Model-View-ViewModel (MVVM)** pattern goes a step further. It adds a **ViewModel** that prepares and manages data for the View.
In MVVM:

- **Model:** It holds the application's raw data and business logic. It manages the data, makes network or database calls, and does not know how the data is shown on the screen.
- **View:** The user interface displays the data on the screen so the user can see and interact with it. The View is kept as simple as possible and contains no business logic. Its job is only to show what the ViewModel provides and to forward user actions.
- **ViewModel:** It sits between the View and the Model. It gets data from the Model and prepares it in a format easy for the View to display. The ViewModel also contains the presentation logic and handles commands and actions from the View, like button clicks. Unlike the Presenter in MVP, the ViewModel does not directly control the View; instead, data binding is often used so that changes in the ViewModel automatically update the View.

A key idea in MVVM is **data binding**. This allows changes in the ViewModel's data to be automatically reflected in the View, without needing extra update code. For example, in frameworks like WPF or Xamarin, you might have a `ViewModel` containing a `UserProfile` with properties such as `Name` and `ProfilePicture`. The View (often defined in XAML) directly binds its text boxes and image controls to these properties. If a user updates their name in the form, data binding also ensures the ViewModel is updated. Likewise, any changes made in the ViewModel, like loading a new profile picture, are instantly shown in the View, all without manually refreshing the UI.
MVVM is especially useful for applications where the user interface wants to stay in sync with the underlying data automatically. It saves you from writing repetitive code to update the UI.

6.5 Comparing MVC, MVP, and MVVM

Although these patterns all share the same goal of separating the user interface, logic, and data, MVC, MVP, and MVVM handle the flow between these parts in slightly different ways.

Pattern	How It Works	Example (Save Button)
MVC	The controller handles user input and updates the Model. When the view needs to show updated information, it pulls data directly from the Model.	User clicks Save → Controller updates Model → View fetches saved data from Model and displays it.

MVP	View sends user actions to the Presenter. The Presenter updates the Model and directly tells the View what to display.	User clicks Save → View calls Presenter → Presenter updates Model → Presenter updates View with new data.
MVVM	View is bound to the ViewModel. The ViewModel updates the Model and automatically keeps the View in sync using data binding.	User clicks Save → ViewModel updates Model → Bound View updates itself automatically with new data.

6.6 When to use architectural design patterns

Architectural design patterns help organize large applications by setting clear rules for how major system parts work together. Here are some examples of when to use some common architectural patterns.

Pattern	When to Use
MVC (Model–View–Controller)	• Great for web apps where user input, data, and UI must be kept separate. • It makes testing and maintaining code easier when the UI and logic change often.
MVP (Model–View–Presenter)	• Good for desktop and mobile apps where the presentation logic needs to be easily testable. • Useful when you want the View to be completely passive.
MVVM (Model–View–ViewModel)	• Best for apps that support data binding, like WPF, Xamarin, or modern frontend frameworks. • Helps automatically keep the UI in sync with the data without extra glue code.

7 Quiz: Design Patterns

7.1 MCQs

Challenge yourself by solving the following quiz questions.

Design Patterns

Question 1: (Select all that apply.)
Which statement is *true* about design patterns?
(Select all that apply.)

- **[CORRECT]** Design patterns provide one of the best solutions, as they have been derived and optimized over time by various experienced programmers.

- **[CORRECT]** Design patterns provide a clean and elegant solution to a large problem by avoiding repetition in code.
- **[CORRECT]** Design patterns can complicate the application's architecture if they are poorly managed.
- It is necessary to use design patterns in the code even if a simpler approach exists.

Question 2: What best describes the design pattern types most commonly used in software products?

- Functional, Relational, Sequential
- **[CORRECT]** Creational, Structural, Behavioral
- Abstract, Concrete, Static
- Object, Method, Class

Question 3: Suppose you are building a notification system that can send emails, SMS, or push notifications based on user settings. Which pattern would you use to create the appropriate notification object at runtime, and why?

- Singleton to ensure that only one notification service exists.
- **[CORRECT]** Factory to create different types of notification objects as needed.
  Explanation: The Factory pattern provides a way to create objects without specifying the exact class. When the type of object needed varies at runtime (email, SMS, push), a Factory can instantiate the right object based on input or configuration, making the system flexible and easily extensible.
- Adapter to make notification classes compatible.
- Composite to combine all notification methods.

Question 4: How does the Singleton pattern differ from other creational patterns?

- It always creates a new object.
- **[CORRECT]** It restricts a class to only one instance, providing a global access point.
  Explanation: The Singleton pattern ensures that a class has only one instance and provides a single global point of access to that instance. This is useful for resources like configuration managers, loggers, or database connections, where having multiple instances could cause problems.
- It allows creating families of related objects.
- It decorates objects with new functionality.

Question 5: You are developing a tool that builds complex HTML documents with optional components (like headers, footers, and multiple sections). Which creational pattern would make your code more flexible and manageable?

- The Adapter pattern

- The Decorator pattern
- The Facade pattern
- **[CORRECT]** The Builder pattern

  Explanation: The Builder pattern is used to construct complex objects step by step, especially when different representations or configurations are needed. It separates the construction logic from the object's representation, making the code easier to maintain and extend when optional or complex components are involved.

Question 6: You have a `Coffee` class. You want to allow adding extra ingredients (like milk, sugar, or whipped cream) dynamically, without changing the base `Coffee` class code. Which structural pattern should you use, and why?

- Proxy to control access to the `Coffee` object.
- **[CORRECT]** Decorator to add new features to `Coffee` objects dynamically.

  Explanation: The Decorator pattern allows you to add new behaviors or features to objects at runtime, by "wrapping" them with additional functionality. This avoids modifying the original class, making it ideal for adding optional features like coffee toppings.
- Composite to treat individuals and groups of `Coffee` objects the same.
- Adapter to convert `Coffee` to another interface.

Question 7: What best describes the Adapter pattern?

- It provides a way to create families of related objects.
- It defines a one-to-many dependency.
- **[CORRECT]** It allows incompatible interfaces to work together by translating one interface into another.

  Explanation: The Adapter pattern acts as a bridge between two incompatible interfaces, allowing existing code to work with new code without modification. It 's commonly used to make old code compatible with new APIs or third-party libraries.
- It restricts object creation to a single instance.

Question 8: You are building a blog platform. When a new article is published, all subscribers should be notified automatically (by email, SMS, etc.), without the blog code having to know the details of each notification method. Which pattern should you use?

- **[CORRECT]** The Observer pattern

  Explanation: The Observer pattern lets you define a subscription mechanism so that multiple objects (observers) can be notified automatically of changes or events in another object (the subject), decoupling the publisher from the subscribers.
- The Singleton pattern
- The Proxy pattern

- The Command pattern

Question 9: What problem does the Command pattern solve, and how is it typically implemented?

- It restricts object instantiation; uses a static method.
- **[CORRECT]** It encapsulates a request as an object, allowing parameterization and queuing of requests.

  Explanation: The Command pattern turns requests or actions into standalone objects, which can then be queued, logged, or undone. This decouples the sender from the receiver and adds flexibility for features like undo/redo, macros, and delayed execution.
- It allows objects to be decorated; uses wrapping classes.
- It allows objects to be iterated; uses `next`/`hasNext` methods.

Question 10: How is data synchronization between the UI and the underlying data model typically achieved in the MVVM (Model-View-ViewModel) architectural pattern?

- Through manual updates in the Controller
- By direct method calls from the View to the Model
- **[CORRECT]** By using data binding, which keeps the View and ViewModel in sync automatically

  Explanation: In MVVM, data binding connects the View and the ViewModel so changes in the ViewModel are instantly reflected in the UI, and vice versa. This automatic synchronization is a key feature, especially in modern frameworks that support declarative UI.
- By letting the Presenter dictate all UI changes